

Objectif du rapport

Je presente ici mon correctif final : un agent RLMinimax qui combine Alpha-Beta avec une fonction affine dont les poids sont appris par renforcement. Le but est de montrer que le code respecte le sujet et que les resultats sont reproductibles.

10/10

layouts meilleurs

+325.3

gain moyen

7

features / poids

0.0

Stop moyen

1. Conformite au sujet

Point	Ce que j ai implemente	OK
Facteurs Pac-Man	7 features liees au jeu : nourriture, fantomes, capsules, score	OK
Recherche	BFS pour calculer des distances reelles dans le labyrinthe	OK
Fonction affine	Evaluation = somme des ai*xi + constante C	OK
Alpha-Beta	Pac-Man MAX, fantomes MIN, coupure alpha >= beta	OK
Poids appris	train.py ajuste les poids par reinforcement learning	OK
Comparaison	compare.py mesure scores, winrate, temps, coups et Stop	OK

Ce que je dois expliquer clairement a l oral

- Le reinforcement learning ne remplace pas Alpha-Beta : il apprend seulement les poids de la fonction d evaluation.
- La fonction respecte le modele affine demande : pas de terme x1*x2, pas de carre, pas de division par une feature.
- Les resultats sont compares avec seed=0 pour garder des conditions identiques entre les agents.

Poids actuels et resultats finaux

Les poids viennent de weights.json et les resultats ont ete relances le 14/05/2026

2. Fonction affine et poids actuels

Modele : $f(s) = a_1 \cdot x_1(s) + a_2 \cdot x_2(s) + \dots + a_7 \cdot x_7(s) + C$

Fact.	Fonction	Poids	Information	Sens
x1	nearest_food_bfs	0.6891	Nourriture proche	Chercher nourriture
x2	ghosts_within_3	-1.4093	Fantomes dangereux	Eviter danger
x3	scared_ghosts_nearby	1.4061	Fantomes effrayes	Profiter opportunités
x4	remaining_food	0.4036	Nourriture restante	Finir la carte
x5	nearest_capsule_bfs	0.0252	Capsule proche	Effet positif faible
x6	current_score	1.8572	Score normalise	Score important
x7	nearest_dangerous_ghost_bfs	-1.2624	Danger principal	Rester loin
C	bias	-0.0834	Correction globale	Constante

Verification : 7 poids pour 7 features, et la valeur calculee correspond a $\sum(a_i \cdot x_i) + C$.

3. Resultats finaux relances

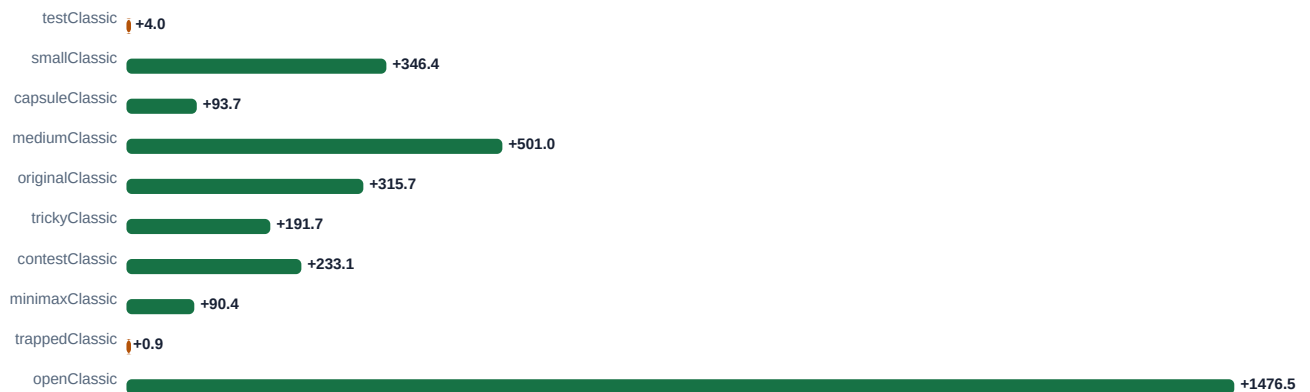
Layout	D	N	Minimax	AlphaBeta	RL	Diff	Win
testClassic	1	200	548.5	548.5	552.5	+4.0	100%
smallClassic	2	100	-149.8	-149.8	196.6	+346.4	34%
capsuleClassic	2	100	-228.9	-228.9	-135.2	+93.7	12%
mediumClassic	1	50	14.5	14.5	515.5	+501.0	28%
originalClassic	1	50	348.6	348.6	664.4	+315.7	2%
trickyClassic	1	50	101.3	101.3	293.0	+191.7	2%
contestClassic	1	50	11.3	11.3	244.4	+233.1	28%
minimaxClassic	3	100	-243.5	-243.5	-153.1	+90.4	34%
trappedClassic	1	1000	-392.3	-392.3	-391.4	+0.9	11%
openClassic	1	10	-419.5	-745.4	1057.0	+1476.5	90%

Lecture : Diff est positive partout, donc RLMinimax obtient un meilleur score moyen sur tous les layouts testes.

Analyse, justification et conclusion

Cette page sert de support pour la defense orale

Gains de score moyen : RLMinimax vs Minimax



Lecture honnete des resultats

- Le winrate n est pas parfait sur toutes les cartes, mais le score moyen est meilleur.
- openClassic est particulier : les baselines dépassent le timeout choisi.
- La conclusion reste solide : les tests montrent un gain statistique en score moyen.

4. Justification algorithmique

Point	Pourquoi c est logique
BFS	Distance reelle dans le labyrinthe, plus pertinente que Manhattan avec des murs.
Alpha-Beta	Meme logique que Minimax, mais moins de branches explorees.
Affine	Le sujet impose une somme ponderee des facteurs, sans termes croises.
Apprentissage	Les poids viennent d un entrainement, pas d un choix manuel arbitraire.
Comparaison	Prouver que les poids appris ameliorent le comportement.

Conclusion finale

Le projet respecte le sujet : fonction affine, facteurs Pac-Man, BFS, Alpha-Beta, poids appris, resultats statistiques et justification claire. Le rapport est pret pour la partie technique ; il reste seulement a verifier le depot Git/Gitesi avant la remise.

Phrase courte pour l oral : mon agent utilise Alpha-Beta pour explorer les coups, puis une fonction affine apprise par renforcement pour evaluer les positions.